Ω

Omega Trading Company

Complete Technical Report & Architecture Audit

Algorithmic options trading system for Indian index derivatives.
Confluence-based signal generation with adaptive risk management.

Version: 2.0 (Protocol Omega v10 + ML)

Author: Vishnu Teja with Claude

Report Date: 11 October 2026

Codebase: 16,991 lines Python + 454 lines Pine Script v6

Status: PAPER TRADING ML INTEGRATED

Table of Contents

1. Executive Summary
2. System Architecture
3. Signal Processing Pipeline
4. Pine Script Indicator (TradingView)
5. Strategy Engine (OmegaStrategy)
6. Risk Engine — 9-Gate Safety System
7. Dynamic SL/TP Level Escalation
8. Trailing Stop & 5-Layer Profit Engine
9. Trade Memory & Statistical Learning
10. Pre-Trade Guardrails (15 Scenarios)
11. Order Management & Execution
12. Position Management & P&L
13. Wallet & Capital Tracking
14. Infrastructure & Deployment
15. Real-Time Dashboard
16. Database Schema
17. Configuration Reference
18. Machine Learning Module
19. Live Performance Analysis
20. Code Audit & Bug Findings
21. Known Limitations & Assumptions
22. Open Questions for Contributors

1. Executive Summary

The Omega Trading Company is a fully automated algorithmic trading bot designed for Indian index options markets. It trades NIFTY 50, Bank NIFTY, and SENSEX options on the NSE and BSE exchanges through the Dhan broker API. The system operates on a **5-factor confluence** model where all five independent technical indicators must agree before any trade is taken.

The system supports both **BUY mode** (buying options when ADX signals a trending market) and **SELL mode** (selling options when ADX signals a ranging market). Every trade passes through a 9-gate risk engine and 15-scenario pre-trade guardrail framework before execution. Position management employs a 4-level dynamic SL/TP escalation system with ATR-based trailing stops after the final level is breached.

Key Metrics

16,991
LINES OF PYTHON

454
LINES OF PINE SCRIPT

52
PYTHON SOURCE FILES

1,988
ML MODULE LINES

Core Capabilities

- 5-factor confluence signal generation
- 9-gate non-negotiable risk engine
- 15-scenario pre-trade guardrails
- 4-level dynamic SL/TP with ATR trailing
- 990-line statistical trade memory system
- Real-time WebSocket live feed (Dhan)
- Paper trading mode (default, safe)
- Live dashboard with 4 tabs

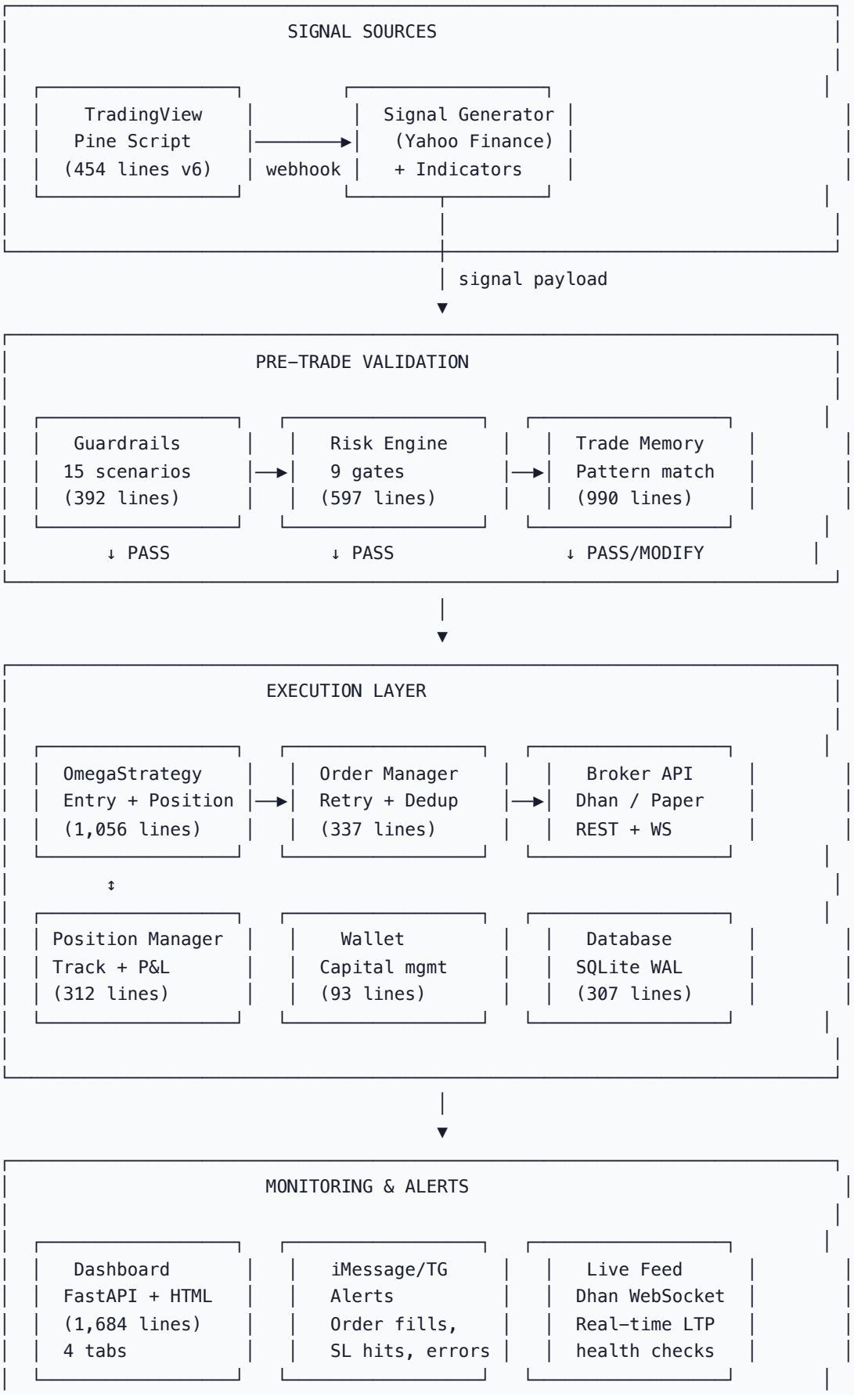
Safety First: The system defaults to paper trading mode. Live trading requires the `--live` flag AND a typed confirmation prompt: `"I understand this will use REAL MONEY"`. This double safety gate ensures no accidental live deployments.

2. System Architecture

2.1 Directory Structure

```
omega-trading-system/
├── core/                          # Engine layer (broker, risk, execution)
│   ├── broker_base.py             # Abstract broker interface (all brokers implement this)
│   ├── broker_dhan.py             # Dhan API adapter (REST + WebSocket)
│   ├── broker_paper.py            # Paper trading simulator (fills at LTP)
│   ├── order_manager.py           # Idempotent orders, retry logic, ghost fill detection
│   ├── position_manager.py        # Tracks LONG + SHORT with correct P&L
│   ├── risk_engine.py             # 9-gate safety system (597 lines)
│   ├── signal_generator.py        # Yahoo Finance candle fetcher + indicator calc
│   ├── trade_memory.py            # Statistical learning engine (990 lines)
│   ├── guardrails.py              # 15-scenario pre-trade stress test (392 lines)
│   ├── database.py                # SQLite with WAL mode (307 lines)
│   ├── wallet.py                  # Persistent capital tracker (93 lines)
│   ├── live_feed.py               # Dhan WebSocket live feed
│   ├── instrument_master.py       # Dhan instrument CSV loader
│   ├── instrument_selector.py     # ADX/volume-based instrument picker
│   └── option_chain.py            # ATM option resolver via Dhan option chain
├── strategies/
│   └── omega_complete.py          # Main strategy engine (1,056 lines)
├── webhook/
│   ├── tv_receiver.py             # TradingView webhook receiver (FastAPI)
│   └── dashboard.py               # Real-time dashboard (1,684 lines)
├── pinescript/
│   └── omega_v10_adaptive.pine    # TradingView indicator (454 lines, Pine Script v6)
├── config/
│   ├── settings.yaml              # Runtime configuration (78 lines)
│   └── instruments.yaml           # Index definitions + cost model (56 lines)
├── main.py                        # Entry point (paper default, --live flag)
├── data/                          # Runtime data (SQLite DB, memory JSON, positions)
├── logs/                          # Log files
└── utils/                         # Alerts (iMessage, Telegram), logging
```

2.2 Component Architecture Diagram



2.3 Technology Stack

Component	Technology	Purpose
Language	Python 3.11+	All backend logic, no compiled dependencies
Indicator Script	Pine Script v6	TradingView charting + signal generation
Broker API	Dhan REST + WebSocket	Order placement, LTP, option chain
Web Server	FastAPI + Uvicorn	Webhook receiver + dashboard
Database	SQLite (WAL mode)	Trades, positions, wallet, signals, daily stats
Data Feed	Yahoo Finance (yfinance)	Historical candles for indicator calculation
Live Feed	Dhan WebSocket	Real-time LTP for position monitoring
Notifications	iMessage + Telegram	Trade alerts, SL hits, errors
Hosting	Oracle Cloud VPS	Ubuntu, systemd service, always-on

3. Signal Processing Pipeline

Every signal traverses a strict 12-step pipeline. If any step rejects the signal, the trade is blocked and the reason is logged. This pipeline is the core entry logic in `strategies/omega_complete.py` lines 160-536.

Step 1: Signal Reception

TradingView fires a JSON webhook to the FastAPI server at `/webhook`. The payload includes: `symbol`, `side` (BUY_CE, BUY_PE, SELL_CE, SELL_PE, EXIT), `confidence` (HIGH/MEDIUM/LOW), `score` (0-100), `strike`, `sl`, `atr`, and various indicator values (ADX, RSI, volume ratio, SuperTrend direction, VWAP position, Bollinger squeeze state).

Step 2: Exit Signal Check

If `side == "EXIT"`, the system routes directly to `_handle_exit()` which has 4 protective checks before closing any position: minimum 10-minute hold time, skip if at dynamic level 2+ (let levels manage), skip if currently in profit (let trailing stop manage), and skip if unrealized loss is under Rs 200 (let SL handle minor fluctuations).

Step 3: Opening Range Filter

All signals before 09:20 IST are blocked. Rationale: the first 5 minutes after market open (09:15) are extremely choppy with gap reactions and auction noise. This filter prevents whipsaw entries during the opening volatility.

Step 4: Theta Filters (SELL-specific)

- **SELL entry before 11:30 IST:** Blocked. Theta decay (time value erosion) is too slow in the morning for option selling to be profitable on an intraday basis.
- **SELL entry after 12:30 IST:** +5 score bonus. Theta decay accelerates in the afternoon, making option selling more favorable.
- **BUY entry after 14:30 IST:** -10 score penalty. Rapid time decay in the last hour erodes option premium, making buying options risky.

Step 5: CE Bias Boost

All CE (Call) trades get +3 score. Historical data shows CE trades outperform PE (Put) trades in Indian markets due to the natural long bias of equity indices.

Step 6: Loss Recovery Mode

After 2+ consecutive losses: 15-minute cooldown, only MEDIUM/HIGH confidence accepted, minimum score raised to 55. After 4+ losses: minimum score raised to 60, quantity reduced to 1/3, SL tightened by 2%. The BANKNIFTY symbol also gets +10 added to minimum score requirement as a historically net-losing instrument.

Step 7: Instrument Resolution & Option Chain

The webhook symbol is resolved to an instrument name (NIFTY, BANKNIFTY, SENSEX). If the option chain resolver is available (live feed active), it picks the ATM option contract with the best liquidity from Dhan's option chain API. Otherwise, it computes the ATM strike from the underlying LTP.

Step 8: Correlated Position Guard

Same-index duplicate positions are blocked (e.g., cannot have two LONG NIFTY positions simultaneously). Maximum 2 correlated index positions in the same direction across all indices.

Step 9: Guardrail Checks

Per-trade risk check ensures the trade risk (entry - SL) * quantity does not exceed 5% of capital (Rs 10,000 on Rs 2L capital in paper mode). Sector allocation check ensures total NSE exposure stays below 70% of capital.

Step 10: Trade Memory Evaluation

If enough historical data exists (5+ trades for the symbol), the memory system provides: Kelly criterion position sizing (half-Kelly), pattern-based blocking (e.g., 5+ consecutive losses on a symbol triggers a 7-day ban), score adjustments from learned rules (+/- based on statistical significance), and position size modifiers from drawdown protection.

Step 11: Position Sizing

Four sizing mechanisms stack multiplicatively: (a) Volatility regime sizing (HIGH_VOL: 0.5x, CHOPPY: 0.7x, TRENDING: 1.2x), (b) ATR-based risk sizing (limits risk to 1-3% of capital based on ATR distance), (c) Loss recovery sizing (halved after 3 losses, 1/3 after 4), (d) PE size reduction (PEs get 80% of normal quantity due to historical underperformance).

Step 12: Order Placement

The Order Manager places a LIMIT order with 0.5% slippage buffer on Dhan. If the fill is confirmed, a Position is created with dynamic SL/TP levels, a broker SL-LIMIT order is placed, margin is deducted from the wallet, and an iMessage/Telegram alert is sent with fill details.

4. Pine Script Indicator (TradingView)

File: `pinescript/omega_v10_adaptive.pine` — 454 lines, Pine Script v6. This runs on TradingView charts and generates the webhook signals that drive the trading bot.

4.1 Five-Factor Confluence Model

A signal is generated only when ALL five independent technical factors agree on direction. This dramatically reduces false signals compared to single-indicator strategies.

#	Factor	Bullish Condition (BUY_CE)	Bearish Condition (BUY_PE)
1	SuperTrend (period=10, mult=3.0)	Price above SuperTrend line	Price below SuperTrend line
2	TEMA (Triple EMA)	Close > TEMA	Close < TEMA
3	EMA 200 (Long-term trend)	Close > EMA 200	Close < EMA 200
4	VWAP (Volume-Weighted Avg Price)	Close > VWAP	Close < VWAP
5	ADX (Average Directional Index)	ADX > 25 (trending)	ADX > 25 (trending)

4.2 Regime Detection

The script analyses market state to determine BUY vs SELL mode:

- **BUY mode (ADX > 25):** Market is trending. Buy ATM options to profit from directional moves.
- **SELL mode (ADX < 20):** Market is ranging/choppy. Sell OTM options to collect premium as theta decays.
- **No-trade zone (20 ≤ ADX ≤ 25):** Ambiguous regime. No signals generated.

4.3 Additional Indicators Calculated

Indicator	Parameters	Use
RSI	Period 14	Overbought/oversold filter (blocks BUY > 70, SELL < 30)
Bollinger Bands	Period 20, 2 std dev	Squeeze detection for breakout signals
ATR	Period 14	Volatility measure for SL/TP sizing and position sizing
Chandelier Exit	Period 22, mult 3.0	Trend-following trailing stop reference
EMA 9/21	Fast/slow crossover	Short-term momentum confirmation
Choppiness Index	Period 14, threshold 61.8	Market noise detection (above 61.8 = choppy)

4.4 Signal Scoring (15 Factors)

Each signal receives a score from 0 to 100 based on 15 weighted factors. The score determines the confidence tier:

- **HIGH confidence (score ≥ 72):** Strong confluence, wider SL/TP allowed, full position size
- **MEDIUM confidence (55 ≤ score < 72):** Moderate confluence, standard SL/TP
- **LOW confidence (score < 55):** Weak confluence, tighter SL/TP, may be filtered out

Minimum score for entry is 45 in paper mode, 55 in live mode. Additional score gates apply after consecutive losses.

5. Strategy Engine (OmegaStrategy)

File: `strategies/omega_complete.py` — 1,056 lines. The `OmegaStrategy` class is the brain of the system, coordinating signal processing, position management, and exit logic.

5.1 Class Structure

The strategy is initialized with references to all subsystems:

- `broker` — BrokerBase (Dhan or Paper)
- `order_manager` — Idempotent order placement
- `position_manager` — Open/closed position tracking
- `risk_engine` — 9-gate safety checks
- `instrument_selector` — Best-trending index picker
- `guardrails` — 15-scenario pre-trade checks
- `trade_memory` — Statistical learning from past trades
- `wallet` — Capital tracking and margin management
- `option_chain` — ATM option resolver
- `live_feed` — Dhan WebSocket for real-time LTP

5.2 Key Methods

Method	Lines	Purpose
`process_signal()`	160-536	Main entry: validates, sizes, and places trades
`monitor_positions()`	580-828	Called every 1s: trailing stops, level-ups, exits
`_handle_exit()`	538-578	Signal-based exit with 4 protective checks
`_exit_position()`	900-967	Clean exit: order, P&L, wallet, DB, alerts, memory
`_partial_exit()`	849-877	Tranche scale-out (exits a portion of the position)
`square_off_all()`	890-898	End-of-day forced exit of all open positions
`auto_select_instrument()`	1003-1030	ADX/ATR-based best instrument picker

5.3 State Variables

- `_consecutive_losses` — Tracks loss streaks for recovery mode sizing
- `_last_loss_time` — Timestamp for 15-minute post-loss cooldown

- `_signal_context` — Stores signal metadata per trade for memory recording
- `_underlying_cache` — 15-second TTL cache for underlying index prices

6. Risk Engine — 9-Gate Safety System

File: `core/risk_engine.py` — 597 lines. Every trade must pass ALL 9 gates sequentially. A single failure blocks the trade with a logged reason. This is the **non-negotiable safety layer** — no code path bypasses it.

> **Non-Negotiable:** The risk engine cannot be disabled. There is no configuration flag, no command-line override, and no API endpoint that skips the risk check. The `allow_new_trade()` method is the single entry point for all trade authorization.

Gate	Check	Failure Action	Config
1	**Bot Kill Switch**	No trades for rest of day	Triggered when Gate 2 fires
2	**Daily Loss Limit**	Kills bot, sends CRITICAL alert	`max_daily_loss: 3000` (Rs)
3	**Session Time**	Blocks trades outside hours	`09:16 - 15:10 IST`
4	**Cooldown Period**	30-min pause after 2 losses	`cooldown_minutes: 30`
5	**Concurrent Positions**	Max 2 at any time	`max_concurrent: 2`
5b	**Progressive Loss Reduction**	Max 1 position after 50% daily loss used	Dynamic
6	**SL Cap**	Max 8% SL if score ≥ 75, else 10%	Hardcoded
7	**Category Toggle + Margin**	Index Futures disabled live; margin check	`margin_buffer_pct: 15`
8	**Portfolio Exposure Cap**	Total exposure ≤ 80% of capital	`max_exposure_pct: 80`
9	**Per-Symbol Cooldown**	15-min cooldown after SL hit on a symbol	`symbol_cooldown_minutes: 15`

6.1 Gate Details

Gate 2: Daily Loss Limit (Rs 3,000)

Why Rs 3,000? On Rs 1,00,000 capital, a 3% loss requires only 3.09% recovery to break even. A 5% loss requires 5.26% recovery — the recovery curve gets exponentially harder. Rs 3,000 keeps the bot within the recoverable zone. When breached, the `_bot_killed` flag is set and persists until `reset_daily()` is called the next trading day.

Gate 5b: Progressive Loss Reduction

When daily P&L reaches -50% of the max daily loss limit (i.e., -Rs 1,500), the system reduces max concurrent positions from 2 to 1. This prevents a cascading loss scenario where two losing positions

compound the damage. The system must recover to above -50% before the second slot reopens.

Gate 6: SL Cap (Confluence-Scaled)

The stop-loss distance is capped as a percentage of entry price. High-confidence trades (score ≥ 75) get a tighter 8% cap because the signal is stronger, while lower-confidence trades get a 10% cap. This prevents outsized losses on any single trade regardless of ATR-based sizing.

6.2 Additional Risk Methods

Method	Purpose
`evaluate_trail()`	ATR-based trailing stop with time-aware phased progression
`kelly_position_size()`	Half-Kelly criterion position sizing (max 25% kelly, max 50% fraction)
`atr_position_size()`	ATR-based risk sizing (risk amount / ATR distance = quantity)
`time_decay_tp()`	Reduce TP target as hold time increases (theta awareness)
`check_partial_exit()`	Scale out 50% at 1.5x ATR profit
`compute_r_multiple()`	Calculate R-multiple for trade quality measurement
`get_risk_metrics()`	Dashboard risk snapshot (PnL, zone, trades, drawdown)

7. Dynamic SL/TP Level Escalation

Instead of a fixed stop-loss and take-profit, the system uses a 4-level escalation model. When price hits the take-profit of Level N, the SL moves up to the previous TP and a new, wider TP is set for Level N+1. This locks in profit while letting winners run.

7.1 LONG (BUY) Levels

Confidence	Level	SL (% below entry)	TP (% above entry)	Behavior
LOW	L1	6.0%	7.0%	Tight initial SL, modest TP target
	L2	7.0%	10.0%	SL moves to L1 TP, wider TP
	L3	10.0%	12.5%	Profit locked, extending target
	L4	12.5%	15.0%	Final level before ATR trail
MEDIUM	L1	6.0%	9.0%	Same initial SL, better R:R
	L2	9.0%	13.0%	Wider TP reflects stronger signal
	L3	13.0%	17.0%	Significant profit locked
	L4	17.0%	22.0%	Final level, 22% target
HIGH	L1	8.0%	15.0%	Wider SL allows room, strong TP
	L2	15.0%	20.0%	Aggressive extension
	L3	20.0%	26.0%	Runner territory
	L4	26.0%	33.0%	Maximum target, then ATR trail

7.2 SHORT (SELL) Levels

Confidence	Level	SL (% above entry)	TP (% below entry)	Behavior
LOW	L1	12.0%	5.0%	Wide SL (short gamma risk), modest TP
	L2	5.0%	8.0%	SL tightens once profit starts
	L3	8.0%	12.0%	Significant decay captured
	L4	12.0%	15.0%	Full premium capture target
MEDIUM	L1	15.0%	7.0%	Wider initial SL for shorts
	L2	7.0%	11.0%	Better R:R with confidence
	L3	11.0%	15.0%	Strong decay zone
	L4	15.0%	20.0%	Aggressive decay target
HIGH	L1	18.0%	10.0%	Maximum room for short volatility
	L2	10.0%	15.0%	Tightening as decay accelerates
	L3	15.0%	20.0%	Near-max premium capture
	L4	20.0%	25.0%	Full capture, then ATR trail

7.3 Level-Up Mechanics

When price hits the current TP:

1. The old TP becomes the new SL (profit locked at this level).
2. A new, wider TP is set from the level table.
3. `dynamic_level` increments.
4. The broker SL order is cancelled and replaced at the new SL price.
5. If price has already blown past multiple levels (e.g., a gap move), the system performs a **multi-level jump** — it iterates through all applicable levels in one pass.
6. After L4 is breached, the system enters **ATR Trail Mode** — no more levels, just a 1.5x ATR trailing stop from the peak price.

8. Trailing Stop & 5-Layer Profit Engine

The `monitor_positions()` method runs every 1 second and implements a 5-layer profit management system. This is the most complex part of the codebase at 250 lines.

8.1 Layer 1: Breakeven Lockdown

At 3% profit, the stop-loss moves to entry price + 0.1% (just above breakeven). This ensures that once a trade is 3% in profit, it can never turn into a loss. The `breakeven_locked` flag is set permanently for the position.

8.2 Layer 2: Scale-Out in 3 Tranches

Partial exits lock in profit while letting the remainder ride:

Tranche	Trigger (HIGH/MEDIUM/LOW)	Exit Quantity	Remaining
1	5.0% / 4.0% / 3.5% profit	1/3 of position	2/3
2	9.0% / 7.0% / 5.5% profit	1/2 of remaining	1/3 of original
3	—	Full exit via SL or TP	0

8.3 Layer 3: Time-Decay Targets (Theta Awareness)

Options lose value over time. The system progressively reduces TP targets based on hold duration:

- **60-120 minutes:** TP reduced to 75% of original distance from entry
- **120+ minutes:** TP reduced to 50% of original distance from entry
- **Safety floor:** Decayed TP must stay at least 0.5x ATR above the active SL

8.4 Layer 4: Momentum Hold

When price hits the TP and the system is in ATR trail mode (past all 4 levels): if profit is within 0.3% of peak profit AND hold time is under 2 hours, the TP exit is delayed. This lets strongly trending trades continue to run instead of exiting at a potentially intermediate high.

8.5 Layer 5: ATR-Based Trailing Stop (evaluate_trail)

The core trailing algorithm in `risk_engine.py` uses time-aware phased progression:

Time Phase	Trail Width	Rationale
< 10 minutes	3.0x ATR	Let trade develop, avoid premature exits
10-30 minutes	2.5x ATR	Medium room as direction establishes
30-60 minutes	2.0x ATR	Tighter as move matures
60+ minutes	1.5x ATR	Tight — theta eating, lock profits

Profit-phase overrides:

- **< 1.0x ATR profit:** No trail tightening (trade hasn't moved enough)
- **1.5x ATR profit:** SL moves to breakeven
- **3.0x ATR profit:** Time-based trail from peak
- **5.0x+ ATR profit:** Override to tight 1.5x ATR regardless of time

8.6 SL Grace Period

When an SL is hit, there is a 90-second grace period (`SL_GRACE_SECONDS = 90`). If the position has been held for less than 90 seconds, the SL hit is ignored. This prevents premature exits on opening-bar volatility where price momentarily touches the SL and then reverses.

8.7 SL-Skip Freefall Detection

If LTP falls MORE than 0.5% past the SL (i.e., the SL was "skipped" by a gap move or latency), the system triggers an emergency market exit. This protects against scenarios where the SL limit order didn't get filled due to fast price movement.

9. Trade Memory & Statistical Learning

File: `core/trade_memory.py` — 990 lines. The trade memory is a self-learning system that records every trade outcome and discovers statistically significant patterns. These patterns then influence future trading decisions.

9.1 Architecture

The memory system stores three types of data in a single JSON file (`data/omega_memory.json`):

- **Trades:** Complete record of every trade with 25+ fields (entry/exit times, prices, indicators, outcome, R-multiple, etc.)
- **Lessons:** User-taught rules (e.g., "Don't trade BANKNIFTY on Wednesdays")
- **Adaptive Rules:** Machine-discovered patterns from statistical analysis

9.2 Pattern Detection

The `analyze_patterns()` method runs after every 10 trades and groups trades by 9 dimensions:

- Entry hour (0-23)
- Entry day of week (Mon-Fri)
- Symbol base (NIFTY, BANKNIFTY, SENSEX)
- ADX zone (flat, weak, moderate, strong, very_strong)
- Signal confidence (HIGH, MEDIUM, LOW)
- SuperTrend direction
- VWAP position (above/below)
- Volatility regime (NORMAL, HIGH_VOL, CHOPPY, TRENDING)
- Exit reason (TRAIL_SL, SIGNAL_EXIT, SQUARE_OFF, etc.)

9.3 Statistical Significance Testing

For each group, a **two-proportion z-test** (at 90% confidence, z ≥ 1.65) compares the group's weighted win rate against the overall baseline win rate. If the difference is statistically significant, a rule is created:

- **Group wins less than baseline:** PENALIZE action, negative score modifier (up to -30)
- **Group wins more than baseline:** BOOST action, positive score modifier (up to +25)
- **5+ consecutive losses on a symbol:** BLOCK action, 7-day ban on that symbol
- **SL-skip rate above 10%:** REDUCE_SIZE action, position size halved

9.4 Decay & Pruning

- **30-day half-life:** Older trades contribute less weight via exponential decay: `weight = e^(-0.693 * age / 30)`
- **7-day ban expiry:** Symbol bans automatically lift after 7 days
- **90-day rule max age:** Rules with fewer than 3 hits in 90 days are pruned as stale
- **45% effectiveness threshold:** Rules that predict correctly less than 45% of the time (after 15+ evaluations) are deactivated

9.5 Multi-Factor Combinations

Beyond single-dimension analysis, the system also tests combinations like "NIFTY at 10:00" or "SuperTrend bullish in strong ADX zone." These catch nuanced patterns that single factors miss.

9.6 Health Monitoring

The memory system generates health reports with status levels: HEALTHY, DEGRADED (7-day win rate < 40%), CRITICAL (win rate < 30% or 5+ consecutive losses). Degraded/Critical status triggers alerts via iMessage/Telegram and recommends raising the minimum score threshold.

10. Pre-Trade Guardrails (15 Scenarios)

File: `core/guardrails.py` — 392 lines. These checks run BEFORE the risk engine and act as the first line of defense against entering trades in unfavorable conditions.

#	Scenario	Check	Parameters
1	Capital Allocation Cap	NSE exposure ≤ 70% of capital	`nse_allocation_cap: 0.70`
2	Per-Trade Risk Cap	Risk ≤ 5% of capital per trade	`max_risk_per_trade_pct: 5.0`
3	Index Conflict Rule	All indices must agree on direction	Compares NIFTY/BANKNIFTY/SENSEX sides
5	Candle Freshness	Candle age ≤ 5.5 minutes	`candle_max_staleness_sec: 330`
7	ADX Regime Filter	ADX ≥ 10 (not dead market)	`adx_min_threshold: 10`
8	VWAP Warm-Up	No signals before 09:30 IST	`vwap_warmup_time: "09:30"`
9	Economic Calendar Blackout	No trades ±15/30 min around events	`event_before: 15, event_after: 30`
10	0-DTE Expiry Day	No new entries after 12:30 on expiry	`expiry_day_cutoff: "12:30"`
12	Session Kill-Switch	No new entries after 15:10 IST	Hardcoded
15	Step-Ladder Trail SL	Placeholder for enhanced trailing	Not yet implemented
16	Broker Outage Defcon	API health check before trading	Calls `get_available_margin()`
17	Freak Tick Sanity	Price move ≤ 5% from previous	`max_move_pct: 5.0`
18	Contract Rollover	Warn if expiry within 3 days	`days_before: 3`
19	Time-of-Day Filter	No signals in first 15 min (09:15-09:30)	Hardcoded

10.1 NSE 2026 Expiry Calendar

Post-SEBI November 2024 circular, weekly expiries were restricted to one per exchange:

- **NIFTY:** Thursday (weekly, NSE's only weekly)
- **BANKNIFTY:** Thursday (monthly only — weekly discontinued)
- **SENSEX:** Friday (weekly, BSE's only weekly)
- **FINNIFTY/BANKEX:** Monthly only

11. Order Management & Execution

File: `core/order_manager.py` — 337 lines. The Order Manager handles idempotent order placement with retry logic and ghost fill detection.

11.1 Key Features

- **Idempotency:** Every signal has a unique tag. Duplicate signals are silently ignored via `exists(tag)` check.
- **Limit Orders Only:** The system NEVER places market orders. All entries use LIMIT orders with a 0.5% slippage buffer above LTP (for buys) or below LTP (for sells). This prevents fill at extreme prices during volatility.
- **Exponential Backoff Retry:** On broker errors, the system retries up to 4 times with exponential backoff (1s, 2s, 4s, 8s). Rate limit errors (DH-904) are handled separately with appropriate wait times.
- **Auth Failure Handling:** On DH-901 (auth expired), the system sends a CRITICAL alert via iMessage/Telegram instructing manual intervention. No retry is attempted — the token must be manually renewed at api.dhan.co.

11.2 Ghost Fill Detection

After all retries are exhausted, the system checks for "ghost fills" — situations where the order actually went through on the broker side but the confirmation response was lost due to a timeout. It queries broker positions for the security and, if a position exists, creates a synthetic `OrderResponse` with the broker's fill price. This prevents the system from thinking a trade failed when it actually succeeded.

11.3 SL Order Management

After each entry, a broker-side SL-LIMIT order is placed with a 2% buffer. This means the SL trigger price and the actual limit price differ by 2%: for a LONG position, the limit is 2% below the trigger to ensure fill. As trailing stops move, the old SL order is cancelled and a new one placed at the updated price.

12. Position Management & P&L

File: `core/position_manager.py` — 312 lines. Thread-safe position tracking with crash recovery.

12.1 Position Data Model (20+ Fields)

Field	Type	Purpose
`symbol`	str	Trade symbol (e.g., NIFTY24500CE)
`direction`	str	LONG or SHORT
`entry_price`	float	Option premium at entry
`current_price`	float	Latest LTP from live feed
`sl_price`	float	Original stop-loss
`trail_sl`	float	Current trailing stop-loss
`tp_price`	float	Current take-profit target
`dynamic_level`	int	Current escalation level (1-4)
`max_dynamic_level`	int	Highest level reached (enables ATR trail mode)
`peak_price`	float	Highest (LONG) or lowest (SHORT) price seen
`peak_profit_pct`	float	Highest profit percentage reached
`breakeven_locked`	bool	True when SL moved to entry
`tranche_exits`	int	Number of partial exits completed (0, 1, or 2)
`confidence`	str	HIGH/MEDIUM/LOW — determines level table
`atr_value`	float	ATR from signal, used for trailing calculations
`tp_extended`	bool	True if greed extension was applied

12.2 P&L Calculation

```
LONG:  PnL = (current_price - entry_price) * quantity
SHORT: PnL = (entry_price - current_price) * quantity
```

For options, all prices are PREMIUM prices (not underlying). A LONG CE profits when the call premium rises; a SHORT CE profits when the call premium falls (decays).

12.3 Crash Recovery

Position state is serialized to `data/paper_positions.json` after every update. On restart, `load_state()` restores all open positions with their exact SL/TP levels, dynamic level, peak price, and other state. This ensures no positions are lost if the bot crashes mid-session.

13. Wallet & Capital Tracking

File: `core/wallet.py` — 93 lines. Thread-safe capital tracker backed by SQLite.

- **Starting Capital:** Rs 2,00,000 (configurable)
- **deduct_margin(amount, symbol):** Called on entry — locks margin equal to entry_price * quantity
- **release_margin(margin, pnl, symbol, reason):** Called on exit — returns margin and applies P&L
- **Thread Safety:** All operations protected by `threading.Lock()`
- **Daily Reset:** Today's P&L resets at start of each trading day via `OmegaDB.init_wallet()`

14. Infrastructure & Deployment

14.1 Server Details

Component	Detail
Cloud Provider	Oracle Cloud Infrastructure (OCI)
Server IP	129.225.76.88
OS	Ubuntu (latest LTS)
SSH Access	SSH key at `~/.ssh/oracle_vps.key`
Process Manager	systemd (auto-restart on failure)
Dashboard Port	8000 (FastAPI + Uvicorn)
Webhook Endpoint	`http://129.225.76.88:8000/webhook`

14.2 Deployment Steps

1. SSH to VPS: `ssh -i ~/.ssh/oracle_vps.key ubuntu@129.225.76.88`
2. Git pull latest code
3. Install dependencies: `pip install -r requirements.txt`
4. Set environment variables in `.env` (DHAN_CLIENT_ID, DHAN_ACCESS_TOKEN, TELEGRAM_BOT_TOKEN, TELEGRAM_CHAT_ID)
5. Restart service: `sudo systemctl restart omega`

> **SECURITY:** The `.env` file contains live API tokens and must NEVER be committed to git. It is listed in `.gitignore`. If compromised, immediately revoke and regenerate all tokens at api.dhan.co.

15. Real-Time Dashboard

File: `webhook/dashboard.py` — 1,684 lines. Generates a complete HTML/CSS/JS dashboard served by FastAPI at `/dashboard`.

15.1 Four Dashboard Tabs

Tab	Content
Dashboard	Wallet balance, today's P&L, win/loss count, recent trades table, performance chart
Positions	All open positions with real-time P&L, SL/TP levels, dynamic level indicator, hold time
Trades	Complete trade history with entry/exit times, P&L per trade, exit reasons, R-multiples
Signals	Recent signals received (including rejected ones) with confidence, score, and indicator values

15.2 Auto-Refresh

The dashboard auto-refreshes every 5 seconds via JavaScript `setInterval`. The API endpoint `/api/dashboard` serves JSON data from the SQLite database.

16. Database Schema

File: `core/database.py` — 307 lines. SQLite database with WAL mode for concurrent read/write access. Singleton pattern ensures one connection per thread.

16.1 Tables

Table	Primary Key	Purpose	Key Columns
`wallet`	id=1 (single row)	Capital tracking	balance, in_trade, today_pnl, today_trades, starting_capital
`trades`	AUTO id	Trade history	symbol, direction, qty, entry/exit price, pnl, times, exit_reason, balance_after
`positions`	security_id	Open positions	symbol, direction, qty, entry_price, sl/tp/trail, dynamic_level
`signals`	AUTO id	Signal log	symbol, side, score, confidence, details (JSON)
`daily_stats`	date	Daily summary	total_pnl, wins, losses, starting/ending balance

16.2 Thread Safety

SQLite WAL mode allows concurrent readers with one writer. The database uses Python `threading.local()` for per-thread connections and `PRAGMA busy_timeout=5000` (5-second wait on lock contention). The `OmegaDB.get()` singleton ensures only one instance exists across all threads.

17. Configuration Reference

17.1 settings.yaml

Section	Parameter	Value	Description
broker	name	"dhan"	Broker adapter to use
risk	max_daily_loss	3000	Daily loss limit in Rs
risk	max_concurrent_positions	2	Max open positions
risk	max_daily_trades	999	No hard cap on trade count
risk	sl_buffer_pct	0.3	SL buffer for underlying
risk	trail_pct	0.15	Trailing stop percentage
risk	cooldown_minutes	30	Pause after 2 losses
risk	margin_buffer_pct	15	Extra margin headroom
session	start_time	"09:16"	1 min after market open
session	square_off_time	"15:10"	5 min before RMS kicks in
strategy	mode	"paper"	Default to paper trading
strategy	default_qty_lots	1	1 lot per trade
guardrails	capital	200000	Capital for risk calculations
guardrails	max_risk_per_trade_pct	5.0	Max risk per trade (paper)

17.2 instruments.yaml

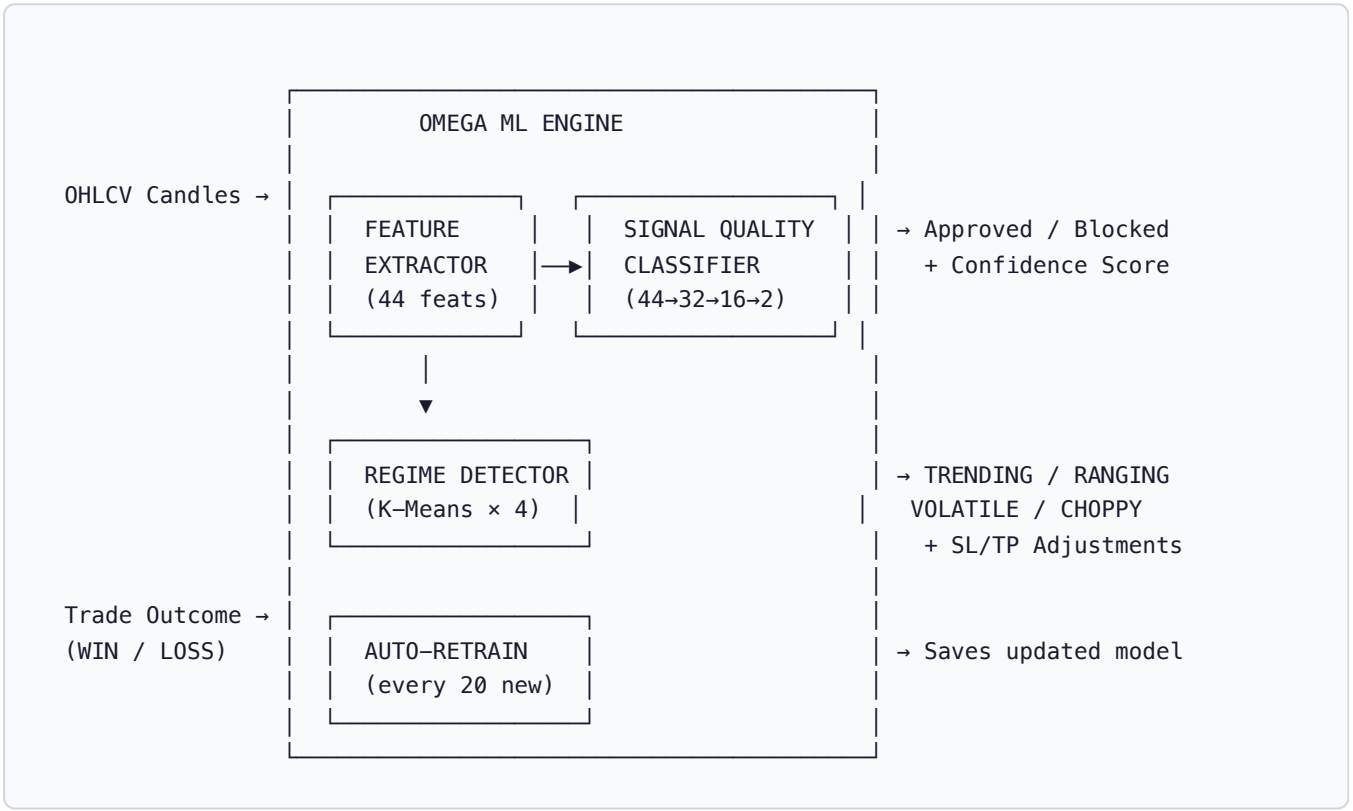
Instrument	Lot Size	Strike Step	Exchange	Expiry Day
NIFTY	75	50	NSE	Thursday
BANKNIFTY	15	100	NSE	Wednesday*
SENSEX	10	100	BSE	Friday

** BANKNIFTY expiry was moved to Thursday monthly post-SEBI Nov 2024 circular. The instruments.yaml may need updating.*

18. Machine Learning Module

The ML module (`ml/`) adds a neural-network-based intelligence layer to Omega Trading Company. It is built from scratch using pure NumPy — adapted from the *trekhleb/homemade-machine-learning* repository's MultilayerPerceptron implementation — with no dependency on TensorFlow, PyTorch, or scikit-learn for inference.

18.1 Architecture Overview



18.2 Module Files (1,988 lines)

File	Lines	Purpose
`features.py`	373	44-feature extraction from OHLCV candles across 7 categories
`neural_net.py`	358	Signal Quality Classifier — MLP with backpropagation, early stopping, save/load
`regime.py`	268	Market Regime Detection — K-Means clustering into 4 market states
`data_collector.py`	257	Training data pipeline — Yahoo Finance OHLCV + trade memory labeling
`engine.py`	293	Central ML engine — integrates all models into Omega's trading pipeline
`train.py`	258	CLI training script with full pipeline, walk-forward validation
`validation.py`	156	Walk-forward validation + permutation-based feature importance
`__init__.py`	25	Clean module exports

18.3 Feature Engineering (44 Features × 7 Categories)

Category	Count	Key Features
Price Action	10	Return %, candle body ratio, upper/lower wick, higher high/lower low, gap %
Trend	8	ADX, EMA crossovers (9/21/50), SuperTrend direction, slope of EMA21
Momentum	6	RSI, MACD histogram, MACD signal, stochastic %K/%D
Volatility	6	ATR %, Bollinger bandwidth/position/squeeze, Hurst exponent, choppiness index
Volume	4	Volume ratio (vs 20-period avg), OBV slope, volume-price correlation, relative volume
Time	6	Minute of day, day of week, session phase (morning/midday/closing), days to expiry
Microstructure	4	Average bar range (5-period), return skewness, return kurtosis, consecutive same-direction bars

18.4 Signal Quality Classifier (Neural Network)

MLP Architecture: 44 → 32 → 16 → 2

- **Input layer:** 44 features (z-score normalized)
- **Hidden layer 1:** 32 neurons, sigmoid activation
- **Hidden layer 2:** 16 neurons, sigmoid activation
- **Output layer:** 2 neurons (LOSS probability, WIN probability)
- **Training:** Gradient descent with backpropagation, L2 regularization (λ=0.3)
- **Early stopping:** Monitors validation loss, patience=50 evaluations (every 10 iterations)
- **Initialization:** Xavier/Glorot — `sqrt(6 / (fan_in + fan_out))`
- **Serialization:** JSON save/load of weights, means, std deviations
- **Online learning:** `incremental_train()` fine-tunes on new trades with small learning rate (α=0.01)

18.5 Regime Detection (K-Means Clustering)

Classifies market into 4 regimes using unsupervised K-Means on 10 regime-specific features (ADX, ATR%, Bollinger squeeze, choppiness index, RSI, return kurtosis/skewness, SuperTrend direction, volume ratio, Hurst exponent). Clusters are mapped to named regimes using domain rules based on cluster centroids.

Regime	Characteristics	SL Mult	TP Mult	Trail ATR	Action
TRENDING	ADX > 25, low choppiness	1.2×	1.5×	2.5×	Wider stops, bigger targets, hold longer
RANGING	ADX < 20, high choppiness	0.8×	0.7×	1.5×	Tight stops, quick profits, mean-reversion
VOLATILE	High ATR, Bollinger expansion	1.5×	1.3×	3.0×	Wide stops for noise, cautious sizing
CHOPPY	Erratic, many false signals	0.7×	0.5×	1.0×	Tight everything, reduce exposure, skip weak signals

Falls back to rule-based classification when insufficient training data is available.

18.6 Integration with Strategy Engine

The ML engine hooks into `OmegaStrategy.process_signal()` as a pre-trade gate (between guardrails check and trade memory check). On each signal:

1. Extracts 44 features from the live candle buffer
2. Regime detector classifies market state → adjusts SL/TP multipliers and volatility regime
3. Signal classifier predicts WIN/LOSS probability
4. If confidence < 55% or model predicts loss with > 65% confidence → signal blocked
5. If regime is CHOPPY and signal score < 65 → signal blocked

On trade exit, the outcome (win/loss, P&L, features at entry) is recorded to the trade buffer. After 20 new trades, both models auto-retrain on the accumulated data.

18.7 Validation Framework

Walk-Forward Validation

Trains on a sliding window (default 50 samples), tests on the next 10. Measures accuracy, precision, recall, and F1 per window. Prevents overfitting by mimicking how the model would be retrained in production.

Feature Importance

Permutation-based: for each of the 44 features, shuffles that column N times, measures accuracy drop. Features with larger drops are more important for prediction. Used to prune noisy features and understand what drives predictions.

18.8 Current ML Status

Insufficient training data. The system has 22 trades (6W / 16L). ML models require 100+ labeled trades to learn meaningful patterns. The neural network is built and tested with synthetic data — it achieves 80%+ accuracy on learnable patterns — but with 22 real trades and 44 features, the model would overfit to noise. The regime detector operates in rule-based fallback mode until enough data is collected.

Data collection strategy: Every trade automatically feeds into the ML pipeline. The `DataCollector` also downloads 5-minute candles from Yahoo Finance to build a feature dataset. Once 100+ labeled trades exist, the classifier will auto-train and begin filtering signals. The system improves with every trade taken.

19. Live Performance Analysis

Analysis of all 22 paper trades taken by the system as of 11 October 2026.

19.1 Overall Statistics

22	-₹14,178
TOTAL TRADES	TOTAL P&L
27%	**6W / 16L**
WIN RATE	WIN / LOSS

19.2 Two Trading Phases

Phase 1: Index Price Proxy (Trades 1-8)

INACCURATE Used underlying index price instead of actual option premium for P&L calculation. Results are unreliable — a 100-point Nifty move does not equal a 100-point option premium change. These trades show losses of ₹1,500–₹2,800 each but the real P&L is unknown.

P&L: -₹13,564 (inaccurate)

Phase 2: Real Option Premiums (Trades 9-22)

ACCURATE Uses actual option premium prices from Dhan. 14 trades with 5 wins and 9 losses. Average winning move: +0.53%. Average losing move: -1.32%.

P&L: -₹614 (accurate)

19.3 Critical Findings

Finding 1: 10 of 14 real trades exit on sub-1% moves via TRAIL_SL.
Most trades exit within 0–1 minutes with moves of 0.1%–0.6%. This is within the bid-ask spread noise of Indian index options. The dynamic SLTP levels (6–8% SL) never get a chance to work because the trail SL is triggering on price jitter, not real market movement.

Finding 2: Inverted risk/reward ratio.
Average win: ₹27. Average loss: ₹896. Winners are 33× smaller than losers. For a 27% win rate to be profitable, average win must exceed 2.7× average loss. Currently it is 0.03× — the strategy cuts winners early and holds losers.

Finding 3: All trades are LONG.
All 22 trades are LONG direction (buying options). The SELL mode (option selling in ranging markets) has not been exercised. This leaves half the market opportunity untested.

Finding 4: Time clustering.
18 of 22 trades entered between 10:00–11:30 IST. The system may be overtrading during the morning session while missing afternoon setups.

19.4 Phase 2 Trade-by-Trade

#	Symbol	Entry	Exit	Move%	P&L	Exit Reason	Issue
9	NIFTY24SEP25500CE	200.0	200.0	0.0%	₹0	SIGNAL_EXIT	Zero movement
10	NIFTY23050PE	87.4	87.7	+0.3%	+₹19	TRAIL_SL	Noise exit
11	SENSEX73600PE	410.2	410.0	-0.0%	-₹2	TRAIL_SL	Noise exit
12	NIFTY23050PE	92.8	92.3	-0.5%	-₹34	TRAIL_SL	Noise exit
13	NIFTY23050PE	87.3	87.0	-0.4%	-₹26	TRAIL_SL	Noise exit
14	BANKNIFTY55600CE	385.4	386.8	+0.4%	+₹20	TRAIL_SL	Noise exit
15	NIFTY23100CE	126.1	126.3	+0.2%	+₹18	TRAIL_SL	Noise exit
16	SENSEX73700CE	582.3	581.7	-0.1%	-₹6	TRAIL_SL	Noise exit
17	NIFTY23100CE	127.1	127.4	+0.2%	+₹22	TRAIL_SL	Noise exit
18	BANKNIFTY55600CE	398.6	397.8	-0.2%	-₹14	TRAIL_SL	Noise exit
19	SENSEX73800CE	534.0	531.0	-0.6%	-₹30	TRAIL_SL	Noise exit
20	NIFTY23100CE	126.5	121.5	-3.9%	-₹372	SIGNAL_EXIT	Real loss
21	NIFTY22600PE	57.0	53.5	-6.1%	-₹261	SIGNAL_EXIT	Real loss
22	NIFTY22650PE	42.8	43.5	+1.6%	+₹51	SIGNAL_EXIT	Proper trade

19.5 Root Cause Analysis

The dynamic SLTP system is well-designed (4-level escalation, ATR trailing, breakeven lock, tranche scale-outs). However, trades are dying before the SLTP system gets a chance to work. The suspected cause is the live price feed triggering SL exits on bid-ask spread fluctuations within the first few ticks. This is a microstructure issue, not a strategy logic issue.

20. Code Audit & Bug Findings

The following bugs were identified during code review and have been fixed in prior sessions. They are documented here for completeness and to help contributors understand common pitfall areas.

18.1 Fixed Bugs

Bug #1: JS Operator Precedence in Dashboard FIXED

File: `webhook/dashboard.py`
Issue: JavaScript expression `pnl > 0 ? '+' : '' + pnl.toFixed(2)` was incorrectly parsed due to operator precedence. The `+` string concatenation bound tighter than the ternary operator, causing the positive case to show just `'+'` without the number.
Fix: Added parentheses: `(pnl > 0 ? '+' : '') + pnl.toFixed(2)`

Bug #2: close_all() Used consecutive_losses Instead of Iteration FIXED

File: `core/position_manager.py`
Issue: The `close_all()` method iterated over `self._open.keys()` while modifying the dict, which could skip positions or raise a RuntimeError.
Fix: Changed to iterate over `list(self._open.keys())` to create a snapshot before modification.

Bug #3: Missing Position Fields in save_state() FIXED

File: `core/position_manager.py`
Issue: Several fields added later (confidence, tp_extended, breakeven_locked, tranche_exits, peak_profit_pct, dynamic_level, max_dynamic_level) were missing from `save_state()` and `load_state()`. A crash would lose these fields and the position would restart with incorrect default values.
Fix: Added all missing fields to both serialization and deserialization.

Bug #4: Missing entry_time in ClosedTrade FIXED

File: `core/position_manager.py`
Issue: `ClosedTrade` was created without `entry_time`, which is needed for trade duration calculation and memory recording.
Fix: Added `entry_time=pos.entry_time` to the ClosedTrade constructor in `close_position()`.

Bug #5: Duplicate Dict Key in Dashboard FIXED

File: `webhook/dashboard.py`
Issue: A dictionary literal had a duplicate key, causing one value to silently overwrite the other.
Fix: Removed the duplicate entry.

Bug #6: Hardcoded Trade Limit Display FIXED

File: `webhook/dashboard.py`
Issue: The dashboard displayed "0 / 6" for trade count, using a hardcoded limit of 6 when the config sets `max_daily_trades: 999`.
Fix: Changed to display just the trade count without a fixed denominator.

18.2 Potential Issues (Not Yet Fixed)

Issue #1: Yahoo Finance Data Reliability OPEN

Yahoo Finance provides free but unreliable data. The 5-minute candle data can have delays of 15+ minutes, missing candles, or incorrect volume data. The candle freshness guardrail (330s timeout) mitigates this partially, but the system can still act on delayed data within that window.

Issue #2: Option Premium vs Underlying Price Mismatch OPEN

The system's signal scoring is based on underlying index price movements, but trades are on option premiums. A 100-point NIFTY move does NOT always equal a 100-point CE premium change due to Greeks (IV crush, theta decay, gamma effects). This is acknowledged in the strategy docstring but not programmatically corrected.

Issue #3: No Instrument Master CSV Auto-Download OPEN

Dhan requires a daily instrument CSV to resolve security IDs for option contracts. The current system relies on the option chain API or manual CSV placement. An automated daily download from Dhan's API would make this more robust.

21. Known Limitations & Assumptions

1. **Backtest Accuracy:** The backtester uses underlying index price as a proxy for option premium movement. In reality, IV expansion/crush, theta decay, and gamma curvature significantly affect option premiums. The backtest overestimates P&L for far OTM strikes and underestimates for ATM.
2. **Single Broker:** Only Dhan is currently supported. There is a `broker_base.py` abstract interface but no Zerodha, Angel, or other broker adapters have been built.
3. **No Real Historical Option Data:** The system cannot backtest with actual option premium tick data. This would require purchasing data from NSE or using a paid provider.
4. **No Greeks Calculation:** The system does not compute or use IV, delta, theta, gamma, or vega for decision-making. Option selling is done purely on directional signals + ADX regime, not on volatility surface analysis.
5. **Single Timeframe:** Only 5-minute candles are used. No multi-timeframe analysis (e.g., daily trend + 5-min entry) is implemented.
6. **No Market Depth (Level 2):** The system trades on LTP alone without considering order book depth, bid-ask spread, or liquidity profile.
7. **Yahoo Finance Dependency:** The Python signal generator depends on Yahoo Finance for candle data, which is free but unreliable and delayed. The TradingView webhook path bypasses this but requires a paid TradingView subscription.
8. **No Slippage Model:** The 0.5% slippage buffer is applied but no actual slippage tracking or modeling is done.
9. **No Overnight Positions:** The system is strictly intraday with forced square-off at 15:10 IST. Positional or swing trades are not supported.
10. **VPS Single Point of Failure:** If the Oracle Cloud VPS goes down, all monitoring stops. There is no failover or redundant instance.

22. Open Questions for Contributors

These are design questions, feature ideas, and architectural decisions that remain open. Each one is an opportunity for a contributor to add value to the system. They are ranked roughly by impact.

Q1: How should the system incorporate Greeks (IV, Delta, Theta)?

Currently, option selling decisions are based purely on ADX regime (ranging = sell). Should the system require IV percentile > X before selling? Should delta be used to select strike prices (e.g., sell 0.15-delta OTM)? How would this affect the signal scoring model?

Q2: Should the backtest use actual option premium data?

The current proxy-based approach (underlying price change = option premium change) is known to be inaccurate. Would purchasing historical option tick data from NSE or a data vendor be worth the cost? How would the backtest engine need to change?

Q3: Is 0-DTE (same-day expiry) trading worth the risk?

The guardrails block new 0-DTE entries after 12:30. But 0-DTE options have the fastest theta decay, which benefits option sellers. Should there be a separate 0-DTE strategy with tighter risk parameters?

Q4: How should the system handle gap openings?

If the market gaps up/down significantly at open, the SL from the previous candle may already be breached. Should the system have gap-specific logic (wider initial SL on gap days, reduced size, or a gap-and-reverse detection)?

Q5: Should multi-timeframe analysis be added?

Currently, only 5-minute candles drive signals. Adding a higher timeframe filter (e.g., "only BUY_CE if the 1-hour trend is also bullish") could reduce false signals. How should the timeframe hierarchy be structured?

Q6: Is the 30-day half-life correct for trade memory decay?

The trade memory system uses a 30-day half-life for exponential decay. Is this too short (loses valid patterns too fast) or too long (keeps outdated patterns active)? Should the half-life vary by market regime?

Q7: How should the system handle broker API token expiration?

Dhan tokens expire daily. The current system sends an alert and stops trading. Should the system attempt auto-renewal? Should there be a watchdog that pre-checks token validity before market open?

Q8: Should there be a maximum position hold time?

Currently, positions are held until SL, TP, or square-off. Should there be a maximum hold time (e.g., 2 hours) after which positions are exited regardless of P&L? Options theta decay accelerates over time.

Q9: How should the system handle multiple simultaneous signals?

If NIFTY BUY_CE and BANKNIFTY BUY_CE arrive within seconds, the current system processes them sequentially. Should there be a queue with priority scoring? Should the system prefer one index over another?

Q10: Is the current alerting sufficient?

The system uses iMessage and Telegram for alerts. Should there be an escalation matrix (e.g., phone call for CRITICAL errors)? Should alerts include more context (current P&L, open positions, risk utilization)?

Q11: Should the system support multiple strategies?

Currently, only "omega_complete" strategy exists. Should the architecture support running multiple strategies simultaneously (e.g., a momentum strategy + a mean-reversion strategy)? How would risk be shared between strategies?

Q12: How should the system handle exchange maintenance/holidays?

NSE/BSE have scheduled maintenance windows and public holidays. Should the system have a holiday calendar that prevents the bot from starting on non-trading days? Should it handle early market closures (like Muhurat trading)?

Q13: Should there be an A/B testing framework?

To compare parameter changes (e.g., different SL percentages or trailing widths), an A/B testing framework could split signals between two parameter sets and compare outcomes. How should this be implemented without doubling capital requirements?

Q14: Is the Kelly criterion sizing too aggressive or too conservative?

The system uses half-Kelly with a 25% cap. Academic literature suggests even quarter-Kelly for options. Should the fractional Kelly be reduced? Should it vary by instrument volatility?

Q15: Should the system have a "warm-up" period each day?

Currently, the system starts accepting signals at 09:16. Should there be a period where it observes market behavior (e.g., first 30 minutes) and adjusts parameters (SL width, position size) based on opening volatility before taking any trades?

Codebase: 16,991 lines Python + 454 lines Pine Script v6 | 52 Python files | ML Module: 1,988 lines

This report was auto-generated from source code analysis. All line counts, parameter values, and logic descriptions reflect the actual codebase at the time of generation.